

Guide 2 — Back-end métier et données

Ta mission : transformer le sujet en données et en règles métier solides — entités, relations, requêtes, données de démo, API de l'orga, IA.

1. Partir du générateur, puis adapter

```
php artisan make:feature PointRegroupement --
fields="nom:string,adresse:string,capacite:integer,ouvert:boolean,latitude:decimal,longitude:decimal" --label="Point de regroupement" --plural="Points de regroupement" --icon=map-pin
php artisan migrate
```

Ensuite, **relis toujours** :

1. La **migration** : index sur les colonnes filtrées, valeurs par défaut, `nullable` corrects.
2. Le **modèle** : relations à ajouter, `casts`, constantes.
3. La **factory** : des données crédibles (c'est ce que le jury verra).
4. La **policy** : les règles par défaut conviennent-elles au sujet ?

2. Relations

```
// Un utilisateur a plusieurs signalements → app/Models/User.php
public function signalements(): HasMany
{
    return $this->hasMany(Signalement::class);
}

// Un signalement concerne une zone → app/Models/Signalement.php
public function zone(): BelongsTo
{
    return $this->belongsTo(Zone::class);
}
```

Migration correspondante :

```
$table->foreignId('zone_id')->nullable()->constrained()->nullOnDelete();
```

Éviter le problème N+1 (une requête par ligne affichée) : charger les relations à l'avance.

```
Signalement::with(['user', 'zone'])->latest()->paginate(10);
Zone::withCount('signalements')->get(); // $zone->signalements_count
```

3. Ajouter une colonne à une entité existante

```
php artisan make:migration add_statut_to_signalements_table
```

```
public function up(): void
{
    Schema::table('signalements', function (Blueprint $table) {
        $table->string('statut', 30)->default('en_cours')->index();
    });
}

public function down(): void
{
    Schema::table('signalements', function (Blueprint $table) {
        $table->dropColumn('statut');
    });
}
```

Puis : l'ajouter au modèle (constante `STATUT_OPTIONS`, et à `#[Fillable]` **seulement** si l'utilisateur a le droit de le modifier lui-même), au formulaire, aux règles de validation, à la factory.

Un statut qui ne doit changer que par un admin (« résolu », « validé ») : il **n'est pas** dans `#[Fillable]`, et on l'assigne dans une action dédiée protégée par la Policy.

4. Requêtes utiles pour les fonctionnalités progressives

```
// Filtres combinés
Signalement::query()
    ->when($niveau, fn ($q) => $q->where('niveau', $niveau))
    ->when($depuis, fn ($q) => $q->whereDate('created_at', '>=', $depuis))
    ->latest()->paginate(10);

// Statistiques (dashboard)
Signalement::count();
Signalement::where('niveau', 'critique')->count();
Signalement::selectRaw('niveau, count(*) as total')->groupBy('niveau')->pluck('total', 'niveau');

// Les plus proches d'un point (approximation suffisante pour une ville)
Signalement::query()
    ->whereNotNull('latitude')
    ->orderByRaw('(POW(latitude - ?, 2) + POW(longitude - ?, 2))', [$lat, $lng])
    ->limit(5)->get();
```

Tester chaque requête sur **MariaDB** en ligne : certaines fonctions SQL diffèrent de SQLite (dates, fonctions mathématiques).

5. Données de démo réalistes

Une application vide paraît inachevée. Chaque entité a une factory et une ligne dans `DatabaseSeeder` :

```
// database/factories/SignalementFactory.php
'titre' => fake('fr_FR')->randomElement([
    'Inondation rue Andrianampoinimerina', 'Arbre tombé à Isoraka', 'Coupure d'électricité à Ivandry',
]),
'niveau' => fake()->randomElement(Signalement::NIVEAU_OPTIONS),
'created_at' => fake()->dateTimeBetween('-10 days', 'now'),
```

```
// database/seeder/DatabaseSeeder.php
\App\Models\Signalement::factory(20)->recycle($users)->create();
```

`recycle($users)` réutilise les utilisateurs existants au lieu d'en créer 20 nouveaux.

En production, pas de `db:seed` complet. Pour remplir l'app en ligne, créer un seeder dédié qui ne crée **aucun compte** :

```
php artisan make:seeder DemoContentSeeder
php84 artisan db:seed --class=DemoContentSeeder --force # sur le serveur
```

6. Consommer l'API fournie par l'orga

Configuration dans `.env` (jamais dans le code) :

```
ORGA_API_URL=https://...
ORGA_API_TOKEN=...
```

```
// config/services.php
'orga' => [
    'url' => env('ORGA_API_URL'),
    'token' => env('ORGA_API_TOKEN'),
],
```

Un service réutilisable, avec cache et gestion d'erreur :

```
// app/Services/OrgaApi.php
class OrgaApi
{
    /** @return array<int, array<string, mixed>> */
    public function alertes(): array
    {
        return Cache::remember('orga:alertes', now()->addMinutes(5), function () {
            return Http::baseUrl(config('services.orga.url'))
                ->withToken(config('services.orga.token'))
                ->acceptJson()
                ->timeout(10)
                ->retry(2, 500)
                ->get('/alertes')
                ->throw()
                ->json('data', []);
        });
    }
}
```

Utilisation : `app(OrgaApi::class)->alertes()` . Si l'API tombe pendant la démo, prévoir un repli (données en base, message clair) plutôt qu'une erreur 500.

Importer en base (fonctionnalité « synchronisation ») :

```
php artisan make:command SyncAlertes
```

```
public function handle(OrgaApi $api): int
{
    foreach ($api->alertes() as $data) {
        Alerte::updateOrCreate(
            ['external_id' => $data['id']],
            ['titre' => $data['title'], 'niveau' => $data['level']],
        );
    }

    return self::SUCCESS;
}
```

Automatiser : dans `routes/console.php` , `Schedule::command('app:sync-alertes')->everyFiveMinutes();` , puis dans cPanel → Tâches Cron, chaque minute :

```
cd ~/webcup-2026 && /opt/cpanel/ea-php84/root/usr/bin/php artisan schedule:run >> /dev/null 2>&1
```

7. Intelligence artificielle (OpenRouter)

La clé ne quitte jamais le serveur. Service type :

```
// app/Services/Ai.php
class Ai
{
    public function ask(string $system, string $prompt): ?string
    {
        $key = 'ai:'.md5($system.$prompt);

        return Cache::remember($key, now()->addHour(), function () use ($system, $prompt) {
            $response = Http::withToken(config('services.openrouter.key'))
                ->timeout(25)
                ->post('https://openrouter.ai/api/v1/chat/completions', [
                    'model' => config('services.openrouter.model'),
                    'messages' => [
                        ['role' => 'system', 'content' => $system],
                        ['role' => 'user', 'content' => $prompt],
                    ],
                ],
            );

            return $response->successful()
                ? $response->json('choices.0.message.content')
                : null; // le composant affiche « Service indisponible, réessayez »
        });
    }
}
```

Dans le composant, **limiter par utilisateur** (sinon quelqu'un vide le quota) :

```
$cle = 'ai:'.auth()->id();
if (RateLimiter::tooManyAttempts($cle, 5)) {
    $this->addError('ia', 'Trop de demandes, réessayez dans une minute.');
```

```
    return;
}
RateLimiter::hit($cle, 60);
```

Garder un modèle gratuit de secours dans `.env` (`OPENROUTER_MODEL=...`) pour pouvoir changer sans redéployer le code.

8. Export CSV

Depuis n'importe quel composant Livewire :

```

public function export()
{
    $this->authorize('viewAny', Signalement::class);

    return response()->streamDownload(function () {
        $out = fopen('php://output', 'w');
        fputcsv($out, ['Titre', 'Niveau', 'Auteur', 'Date']);
        Signalement::with('user')->latest()->chunk(200, function ($lignes) use ($out) {
            foreach ($lignes as $s) {
                fputcsv($out, [$s->titre, $s->niveau, $s->user?->name, $s->created_at-
                >format('d/m/Y')]);
            }
        });
        fclose($out);
    }, 'signalements.csv');
}

```

Bouton : `<flux:button wire:click="export" icon="arrow-down-tray">Exporter</flux:button>`.

9. E-mails et notifications

- Local : `MAIL_MAILER=log` → les e-mails s'écrivent dans `storage/logs/laravel.log`.
- Serveur : `MAIL_MAILER=sendmail` (à tester avant le jour J).

```

php artisan make:notification SignalementCritique
php artisan make:notifications-table && php artisan migrate # pour les notifications dans
l'app

```

```

public function via(object $notifiable): array { return ['database', 'mail']; }

```

Envoi : `$admin->notify(new SignalementCritique($signalement));`

10. Tests à écrire pour chaque règle métier

```

test('un signalement critique notifie les admins', function () {
    Notification::fake();
    $admin = User::factory()->admin()->create();

    Livewire::actingAs(User::factory()->create())
        ->test('pages::signalements.form')
        ->set('titre', 'Inondation')
        ->set('niveau', 'critique')
        ->call('save');

    Notification::assertSentTo($admin, SignalementCritique::class);
});

```

Utilise `/fonctionnalite` dans Claude Code : il écrit les tests en même temps que le code.